

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Recent deep-learning optimizers, including Schedule-Free methods, AdEMAMix, MARS, and Lion, revise the coupling between accumulated momentum and the current stochastic gradient. A complementary theoretical literature establishes that stochastic gradient descent (SGD) can be accelerated in noise-dominated regimes by decoupling the momentum coefficient from the coefficient assigned to the current gradient. This paper gives a unified optimizer-level account of these developments. We show that several recent optimizers instantiate, approximate, or augment accelerated-SGD-style updates, typically through preconditioning, averaging, or coordinate-wise transformations, and we test this interpretation on a 150m-parameter language-modeling problem. In the small-batch regime studied here, AdEMAMix—the evaluated optimizer whose update most closely matches accelerated SGD variants—achieves the strongest performance. The same analysis yields Simplified-AdEMAMix, which preserves AdEMAMix-level performance in both the small- and large-batch experiments while removing the need to maintain two distinct momentum terms. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Recent optimizer design for deep learning has produced methods such as Lion (Chen et al., 2023), Schedule-FreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although these algorithms were introduced from different motivations, their update rules share a central structural feature: each changes how accumulated momentum is combined with the current stochastic gradient.

A parallel theoretical literature studies acceleration for gradient methods in noisy regimes. Classical heavy-ball and Nesterov momentum accelerate deterministic gradient descent, especially on quadratic objectives, but they do not by themselves accelerate SGD (Jain et al., 2018; Liu & Belkin, 2020). This limitation motivates alternative stochastic momentum schemes (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). Across these schemes, the operative mechanism can be expressed as a decoupling between the coefficient governing the momentum state and the coefficient multiplying the current gradient in the parameter update.

This paper connects the empirical optimizer literature and the theoretical acceleration literature through a single optimizer-level account. We prove that Schedule-Free SGD is mathematically equivalent to accelerated SGD followed by weight averaging. We further show that Lion, Schedule-Free AdamW, MARS, and AdEMAMix can be interpreted as accelerated-SGD-style updates combined with additional operations, including preconditioning, averaging, and coordinate-wise transformations. Although prior work has identified related momentum–averaging structure (Defazio, 2021), the explicit relationship between these recent optimizers and accelerated SGD variants has not, to our knowledge, been established in this form.

We evaluate the implications of this analysis on a 150m-parameter decoder-only transformer trained for 15b tokens with a 32k-token batch size, a configuration selected to emphasize stochastic-gradient noise. The empirical comparisons support the theoretical account: Schedule-Free AdamW closely tracks accelerated-SGD-based AdamW (Algorithm 3), and accelerated variants provide modest improvements in this small-batch setting. Complementary large-batch experiments show that these advantages diminish relative to Adam, consistent with the interpretation that the acceleration mechanism is most useful when stochastic noise is substantial.

The paper makes five connected contributions:

1. We establish precise theoretical connections between accelerated SGD and recently proposed optimizers, including Schedule-Free SGD and AdEMAMix.
2. We validate the analysis on a 150-million-parameter decoder-only transformer by comparing AdamW, Schedule-Free AdamW, AdEMAMix, MARS, and related variants; in these experiments, AdEMAMix, whose update most closely aligns with accelerated SGD variants, achieves the best performance among the evaluated methods.
3. We show that the performance advantages of accelerated-SGD-related methods diminish at large batch sizes relative to Adam, and that Adam with momentum scheduling can match AdEMAMix in this regime.
4. We identify a high-batch-size failure mode of Schedule-Free AdamW: it performs substantially worse than AdamW with cosine decay, a gap that we attribute to the intrinsic coupling between momentum and weight-averaging coefficients in Schedule-Free optimizers.
5. We introduce Simplified-AdEMAMix, a modification of AdEMAMix that preserves its performance across the evaluated small- and large-batch regimes while eliminating the requirement to maintain two distinct momentum terms.

2 RELATED WORK

We review the strands of accelerated stochastic optimization and modern optimizer design that supply the technical context for the equivalences and interpretations developed in this paper.

Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for least-squares objectives. Kidambi et al. (2018) simplified the corresponding update and formally showed that standard momentum does not provide acceleration in that setting. Subsequent work (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended accelerated stochastic-gradient guarantees to general convex and strongly convex objectives under different theoretical assumptions, thereby isolating momentum structures that are distinct from classical heavy-ball and Nesterov updates.

A number of practical optimizers exhibit closely related update structure. Lucas et al. (2019) proposed an update based on a weighted combination of multiple momentum terms with distinct coefficients. Ma & Yarats (2019) developed an optimizer explicitly motivated by the theoretical framework of Jain et al. (2018). More recently, Chen et al. (2023) introduced a genetically searched optimizer that, like accelerated SGD variants, assigns different weights to the gradient and to the momentum term in the update. Pagliardini et al. (2024) likewise combines two momentum scales in the final parameter update. These designs motivate a systematic comparison between modern optimizer rules and the decoupled-gradient structure that characterizes accelerated stochastic methods.

3 BACKGROUND

3.1 MOMENTUM

Momentum is the standard mechanism for accelerating gradient descent in deterministic optimization. For parameters w_t and momentum coefficient β , the canonical update is:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging reduces the variance of stochastic iterates by returning an aggregate point along the optimization trajectory rather than the terminal iterate. Instead of using w_T , the procedure constructs a weighted average $\bar{w}_T$, with averaging coefficients denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

Throughout the paper, the relevant weight-averaging schemes use coefficients γ_t with the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for a constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

We represent the accelerated SGD variants considered in the paper using the following common parameterization:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are scalar coefficients that may vary with time, and g_t is the stochastic gradient evaluated at w_t . The subscript ‘a’ identifies coefficients associated with this accelerated-SGD parameterization.

Setting $\alpha_{a,t} = 0$ recovers standard SGD with momentum, whereas nonzero $\alpha_{a,t}$ explicitly separates the current-gradient weight from the momentum coefficient. As detailed in Appendix B, the accelerated SGD algorithms proposed by Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) can all be written in this form.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

We now use the accelerated-SGD form to characterize several recently proposed optimizers. The analysis shows that Schedule-Free methods, Lion, MARS, and AdEMAMix share the same decoupled-momentum core, while also identifying the additional operations—including averaging, preconditioning, and coordinate-wise transformations—that distinguish the practical algorithms from the bare accelerated-SGD template. This perspective motivates a simplified AdEMAMix variant that uses a single momentum term while retaining AdEMAMix-level performance in the evaluated small- and large-batch regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a constant-learning-rate optimizer designed to eliminate the need for an explicit learning-rate schedule. Using the notation of Defazio et al. (2024), its update is:

$$\begin{aligned} y_t &= (1 - \beta) z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1}) x_t + c_{t+1} z_{t+1} \end{aligned}$$

Here, y_t denotes the training weights at which the gradient is evaluated, whereas x_t denotes the weights used for evaluation.

To expose the accelerated-SGD structure, we rewrite the dynamics in terms of y_t and an auxiliary momentum variable m_t , defined by:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Expanding this definition yields the recurrence:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus, m_t follows the momentum update in Equation (1) with $\beta_{a,t} = 1 - c_t$. Given this auxiliary momentum, the update for y_t is:

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Thus, y_t follows the weight update in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, where $w_t = y_t$. The training iterate of Schedule-Free SGD therefore exactly instantiates the accelerated-SGD framework. Because Schedule-Free SGD evaluates with x_t , we also characterize the dynamics of the evaluation iterate:

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

Thus, x_t is a weighted average of the accelerated-SGD iterates y_t . A recursive expansion shows that, when c_t is constant, x_t is an exponential average of y_t . Schedule-Free SGD is consequently not merely analogous to accelerated SGD: it is accelerated SGD followed by weight averaging.

This decomposition separates the advantages of Schedule-Free SGD into two identifiable components:

1. Improved performance relative to standard SGD with momentum, attributable to the accelerated-SGD component.
2. The use of a constant learning rate without explicit scheduling, enabled by weight averaging, and specifically by tailed weight averaging as discussed in Section 4.1.3.

The Schedule-Free construction also provides two implementation-level advantages:

- It performs weight averaging without additional memory.
- It avoids introducing a separate weight-averaging coefficient as a hyperparameter.

As shown in Section 5.1, however, the same coupling between the momentum and weight-averaging coefficients becomes limiting at large batch sizes.

4.1.1 CASE: $\beta = 0.0$

As noted in (Defazio et al., 2024), setting $\beta = 0$ reduces Schedule-Free SGD to standard SGD with weight averaging. Because $c_t = 1/t$, the averaging window begins at the start of training.

4.1.2 CASE: $\beta = 1.0$

As noted in (Defazio et al., 2024), setting $\beta = 1$ reduces Schedule-Free SGD to standard momentum SGD, with momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

For the default Schedule-Free SGD setting $\beta = 0.9$, the accelerated-SGD interpretation gives the following parameter-level consequences:

- Since c_t scales as $1/t$, the effective momentum grows as $1 - 1/t$.
- The ratio between the current-gradient weight and the momentum weight is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- Weight averaging is effectively applied over approximately the most recent 10% of the iterates.

4.2 LION

Lion (Chen et al., 2023) uses the following update rule:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

This update gives a direct interpretation of Lion (Chen et al., 2023) as an accelerated-SGD-style momentum construction followed by a coordinate-wise sign operation. Lion therefore preserves the decoupled gradient-momentum weighting while replacing the raw vector update by a sign-transformed direction.

4.3 MARS

The practical version of MARS (Yuan et al., 2024), referred to as MARS-Approx, also fits the accelerated-SGD template once its preconditioning step is made explicit. Ignoring bias correction and clipping, its update is:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t are the first- and second-order momentum terms, respectively, and x_t denotes the model parameters. Introducing $\hat{m}_t = m_t - \gamma g_t$ gives the equivalent representation:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

This representation shows that the momentum component follows the general accelerated-SGD framework, while the preconditioner is formed differently from AdamW. We therefore treat MARS as an accelerated-SGD-related optimizer whose empirical behavior also reflects its distinct preconditioning rule, and we evaluate it in Section 5.

4.4 ADEMAMIX

AdEMAMix (Pagliardini et al., 2024) is structurally closest to accelerated-SGD-based AdamW among the optimizers considered here. Rather than combining the current gradient with a single momentum term, as in the accelerated-SGD parameterization, AdEMAMix maintains two momentum terms with different coefficients and forms a linear combination of them. Algorithm 1 states the update formally.

For analysis, we first consider the AdEMAMix variant with $\beta_1 = 0$. Pagliardini et al. (2024) showed that this variant nearly matches the full method at small batch sizes, and the experiments in Section 5 confirm the same pattern. Under $\beta_1 = 0$, AdEMAMix aligns with the accelerated-SGD form in Equation (1). Its prescribed schedules for β_3 , the slow-momentum coefficient, and α , the coefficient controlling the relative contribution of the current-gradient component, also closely match theoretical schedules proposed for accelerated SGD (Gupta et al., 2023).

In smooth convex optimization, stochastic acceleration requires a momentum schedule of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximately realizes this schedule by increasing β_3 over training.

The relationship is also visible in the normalization of the momentum state. Accelerated SGD maintains momentum in the form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas Algorithm 1 uses the EMA-style update:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients in accelerated SGD grows proportionally to t . The AdEMAMix coefficient α follows the same proportional scaling. This correspondence explains why AdEMAMix is the evaluated optimizer most closely aligned with accelerated-SGD variants and helps interpret its leading performance in Figure 1.

At large batch sizes, however, AdEMAMix degrades when $\beta_1 = 0.0$, as also reported by Pagliardini et al. (2024). The analysis of Gupta et al. (2023) suggests that the coefficient on the current gradient should decrease as batch size increases. AdEMAMix instead assigns the current gradient a fixed weight of 1, a mismatch that likely contributes to the observed large-batch degradation.

These observations motivate the simplified variant introduced next: it explicitly weights the current gradient, removes the need to schedule α , and maintains only one momentum term. The experiments show that this reduced update matches AdEMAMix in both the small- and large-batch settings considered here.

4.5 SIMPLIFIED ADEMAMIX

The equivalences above lead directly to Simplified-AdEMAMix, an optimizer that removes AdEMAMix’s second momentum term and replaces the scheduled coefficient α with a fixed current-gradient weight. Algorithm 2 presents the method using theory-style momentum rather than EMA-style momentum. The final update assigns the current gradient coefficient α ; setting $\alpha = 0$ recovers standard Adam after the corresponding transformations of η and β_1 . The experiments in Section 5 show that this simplified update matches AdEMAMix across both evaluated batch-size regimes, making the theoretical connection operational as a lower-memory optimizer design.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

-
- 1: Sample batch B_t .
 - 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
 - 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
 - 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{\hat{v}} + \epsilon}$
 - 5: $w \leftarrow w - \eta N$
 - 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
 - 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
 - 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$
-

5 EXPERIMENTS

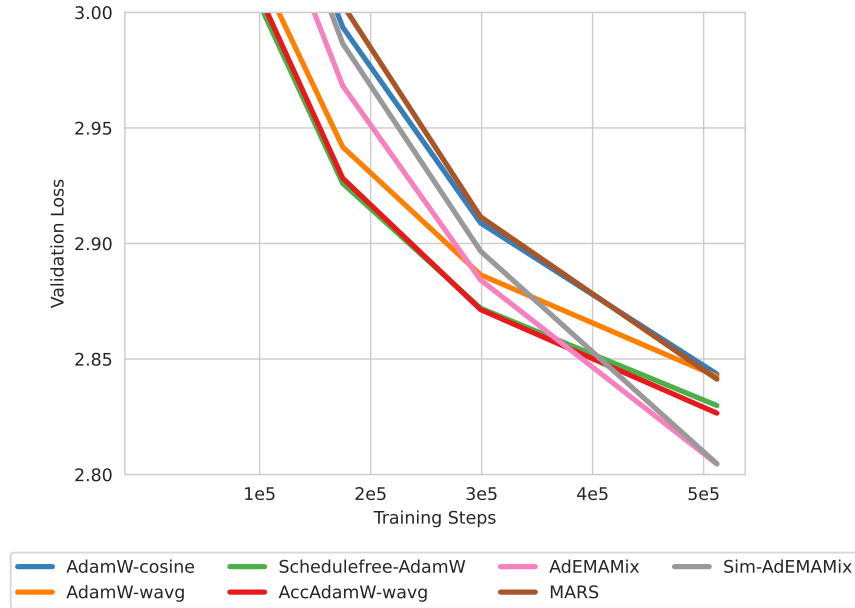


Figure 1: Best-run comparison for the optimizers evaluated in Section 5 on language modeling with a 150m-parameter decoder-only transformer. AdEMAMix and Simplified-AdEMAMix achieve the strongest performance among the evaluated methods, consistent with their close correspondence to accelerated SGD variants.

We evaluate the theoretical connections on language modeling with a 150-million-parameter decoder-only transformer trained on the C4 dataset. The model uses sequence length 1024 and batch size 32, and it is trained for 15 billion tokens ($\approx 5 \times$ Chinchilla). This configuration targets the noise-dominated regime in which the accelerated stochastic mechanisms identified above are expected to be most consequential.

The evaluation compares the following optimizer families and variants, chosen to separate standard AdamW behavior, schedule-free averaging, accelerated-SGD-style updates, and two-momentum designs:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)

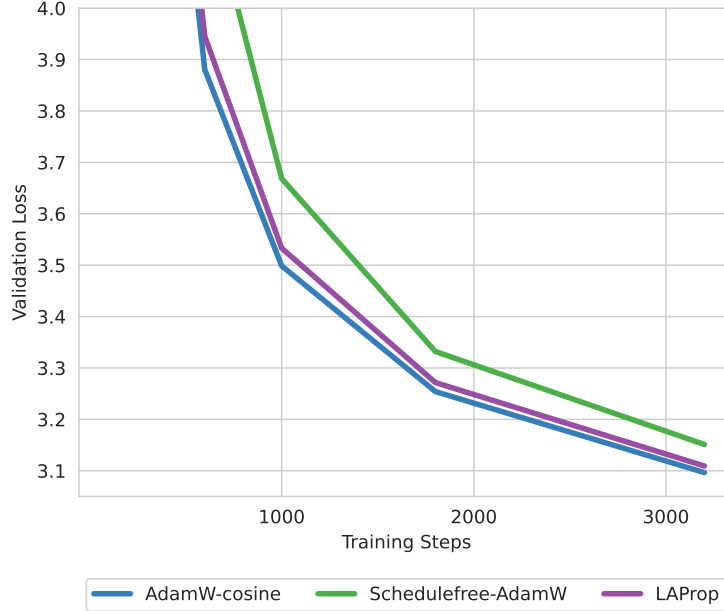


Figure 2: Best-run comparison of AdamW with cosine decay, Schedule-Free AdamW, and LARProp at higher batch size. The comparison isolates whether Schedule-Free AdamW’s large-batch behavior is explained by preconditioning order or by the coupling between momentum and weight averaging. Experimental details are provided in Section 5.1.

5. MARS

6. AdEMAMix

7. Simplified-AdEMAMix

The hyperparameter sweeps used for each optimizer are reported in Appendix A.

Figure 1 presents the small-batch comparison implied by the theory. Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging obtain comparable performance, supporting the derived equivalence between schedule-free training and accelerated SGD followed by averaging. Both methods outperform AdamW with cosine decay and AdamW with tailed weight averaging. AdEMAMix and Simplified-AdEMAMix achieve the best performance among all evaluated optimizers, consistent with their closer alignment with accelerated SGD variants.

5.1 LARGE BATCH SIZE EXPERIMENTS

The preceding experiments target the small-batch, high-noise regime. We next evaluate the same optimizer family at larger batch size to determine which empirical advantages persist when gradient noise is reduced. In this setup, models are trained with a batch size of 1 million tokens for 3 billion tokens ($\approx$ Chinchilla scale).

Schedule-Free AdamW: Figure 2 shows that Schedule-Free AdamW performs substantially worse than AdamW in the large-batch setting. We attribute this gap to the coupling between weight-averaging and momentum coefficients. At higher batch sizes, the preferred momentum value is substantially below $1 - 1/t$. A scaling factor of approximately $1 - r/t$ for some $r \geq 1$ can reduce the effective momentum, but increasing r also shortens the effective weight-averaging window, thereby weakening the averaging component that supports constant-learning-rate training.

AdamW and Schedule-Free AdamW also differ in the order of momentum and preconditioning. AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW applies preconditioning before momentum, making it algorithmically similar to LARProp (Ziyin et al., 2021). In Figure 2, however, AdamW and LARProp perform comparably, indicating that this ordering difference is unlikely to be the principal source of the Schedule-Free AdamW gap.

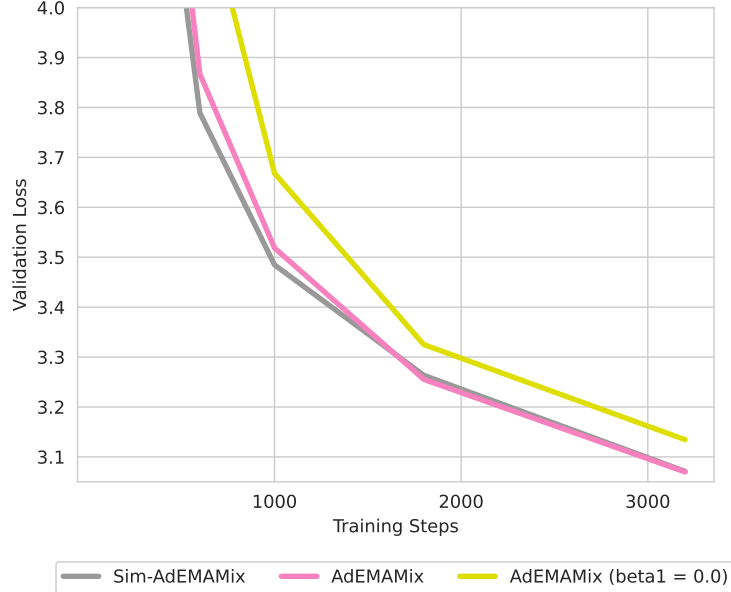


Figure 3: Best-run comparison of AdEMAMix, AdEMAMix with $\beta_1 = 0.0$, and Simplified-AdEMAMix in the higher-batch-size experiments. The figure tests whether the accelerated-SGD interpretation remains predictive when gradient noise is reduced. Experimental details are provided in Section 5.1.

AdEMAMix: In the large-batch experiments, Figure 3 reproduces the pattern reported by Pagliardini et al. (2024): setting $\beta_1 = 0.0$ causes a substantial drop relative to using two separate momentum terms. This degradation is consistent with the accelerated-SGD interpretation. AdEMAMix assigns the current gradient a fixed weight of 1, whereas theoretical accelerated SGD variants (Gupta et al., 2023) require this weight to decrease as batch size increases.

Figure 3 also shows that Simplified-AdEMAMix matches AdEMAMix while removing the second momentum term. In this setting, the matching performance is obtained at $\alpha = 0.0$, so Simplified-AdEMAMix reduces to standard Adam with momentum scheduling.

6 CONCLUSION

This work establishes explicit connections between accelerated SGD variants and several recent optimizers, including Schedule-Free optimizers, AdEMAMix, MARS, and Lion. The analysis identifies a common decoupled-momentum structure underlying these methods, while the experiments show that AdEMAMix, the evaluated method most closely aligned with theoretical accelerated SGD variants, performs best in the small-batch training regime.

The same connection motivates Simplified-AdEMAMix, which removes the need to maintain two momentum buffers. Across the evaluated small- and large-batch settings, Simplified-AdEMAMix matches AdEMAMix while eliminating the additional momentum-memory overhead, making the accelerated-SGD interpretation operational as a simpler optimizer design.

REFERENCES

Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.

- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

The small-batch experiments use the following hyperparameter sweeps, which define the optimization conditions for the comparisons in Section 5.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 were .9 and .999, respectively, matching the default values. Although larger-batch training commonly uses $\beta_2 = .95$, the benefit of higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ i.e. we fix β_1, β_2 to be the optimal values from the previous sweep. This configuration performed worse than the 51.2k-step warmup.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The large-batch experiments use the following hyperparameter sweeps, corresponding to the reduced-noise setting analyzed in Section 5.1:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

Equation (1) gives the general accelerated-SGD form used in the main text. This appendix records the algebraic rewritings that place the methods of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) within that shared parameterization.

B.1 AGNES

The update of Gupta et al. (2023) is:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n , and the final function value is evaluated at x_n . These equations admit the equivalent form

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} follows the accelerated-SGD update form in Equation (1).

B.2 ASGD

The update of Jain et al. (2018) is:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} , and the function is evaluated at the tail-averaged x .

The same dynamics can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

The resulting equations follow the common form in Equation (1).

B.3 MASS

The update of Liu & Belkin (2020) is:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t , and the function is evaluated at w_t . These equations can be rewritten as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The resulting equations follow the common form in Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update of Vaswani et al. (2019) is:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations can be rewritten as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$